Task 10: Research Topics & Bonus Report

Part 02: Research Topics

1. Parallel Programming and Concurrency

Concurrency vs. Parallelism

- **Concurrency:** Managing multiple tasks at once by interleaving execution (context switching) on single or multi-core systems.
- **Parallelism:** Executing multiple tasks simultaneously across physical CPU cores to increase hardware throughput.

Key Tools in .NET

- **Task Parallel Library (TPL):** Simplifies CPU-bound parallel workloads using `Parallel.For`, `Parallel.ForEach`, and `Task.Run`.
- **PLINQ (Parallel LINQ):** Automatically parallelizes LINQ query execution via `.AsParallel()`.

Concurrency Challenges & Synchronization

- **Race Conditions & Deadlocks:** Occur when threads access shared resources without synchronization.
- **Constructs:** Handled using `lock` statements, `Monitor`, `Mutex`, and thread-safe collections (e.g., `ConcurrentDictionary`).

2. Unit Testing and Test-Driven Development (TDD)

Unit Testing & AAA Pattern

- **Arrange:** Setup the test environment, objects, and dependencies.
- **Act:** Execute the target method under test.

- **Assert:** Verify that actual results match expected outcomes.

Frameworks & Mocking

- **Frameworks:** xUnit, NUnit, MSTest.
- **Mocking Tools:** Moq and NSubstitute for isolating code from external dependencies.

TDD Cycle (Red-Green-Refactor)

- **Red:** Write a failing test defining required functionality.
- **Green:** Write minimal implementation code to pass the test.
- **Refactor:** Clean and optimize code while maintaining passing tests.

3. Asynchronous Programming with async and await

Purpose

Prevents thread blocking during I/O-bound operations (database queries, network requests), keeping applications responsive and scalable.

Keywords & Return Types

- `async` and `await` handle control flow without blocking threads.
- Return types include `Task` (for void), `Task<T>` (for returning values), and `ValueTask` (for high-performance scenarios).

Part 03: Bonus

1. Self-Study Report

During Task 10, key advanced C# concepts were explored and applied, specifically focused on functional programming constructs in object-oriented environments[cite: 1]. Core takeaways include:

- Mastered explicit dynamic sorting algorithms using generics, custom comparers, and delegates[cite: 1].
- Understood internal mechanics of built-in delegates (`Func` , `Action` , `Predicate`) and lambda expressions[cite: 1].
- Researched enterprise architecture practices including TDD, Concurrency, and Asynchronous programming patterns[cite: 1].

2. What is Asynchronous Programming?

Asynchronous Programming is a design pattern that enables non-blocking operations[cite: 1]. When an app requests data from an external resource (e.g., File I/O or Web API), the calling thread does not sit idle waiting for a response[cite: 1]. Instead, it yields control back to the system thread pool to process other user interactions or requests, resuming execution seamlessly once the background operation completes[cite: 1].